

Operating System

Complete MCQ-Oriented Notes

IBPS SO (IT OFFICER)
COMPLETE PREPARATION SERIES

100% SYLLABUS COVERAGE
CONCEPT-FOCUSED LEARNING
MCQ-ORIENTED APPROACH
QUICK REVISION READY

COMPILED BY
Asabhya Maanav

2026 EDITION
VERSION 1.0

Table of Contents

Operating System — Complete Notes	4
1. Introduction to Operating Systems	4
1.1 Basics of an Operating System	4
1.2 Operating System Structures	5
2. Types of Operating Systems	6
2.1 Classification of Operating Systems	6
3. Process Management	7
3.1 Process Concepts	7
3.2 Process Scheduling	8
3.3 Threads	9
3.4 Interprocess Communication (IPC)	10
4. CPU Scheduling	11
4.1 Basic Concepts	11
4.2 Scheduling Algorithms (with Gantt Charts)	11
4.3 Advanced Scheduling	13
5. Process Synchronization	14
5.1 Synchronization Basics	14
5.2 Software and Hardware Solutions	14
5.3 Semaphores	15
5.4 Classical Synchronization Problems	15
5.5 High-Level Synchronization Constructs	16
6. Deadlocks	16
6.1 Deadlock Concepts	16
6.2 Deadlock Handling Methods	17
7. Memory Management	19
7.1 Background	19
7.2 Contiguous Memory Allocation	19
7.3 Paging	20
7.4 Segmentation	21
8. Virtual Memory	21
8.1 Virtual Memory Concepts	21
8.2 Page-Replacement Algorithms	22
8.3 Frame Allocation and Thrashing	23
9. File Management	24
9.1 File Concepts	24
9.2 Directory Structure	24

9.3 File Allocation Methods	24
9.4 Free-Space Management	25
9.5 Protection and Access Control	25
10. I/O and Disk Management	25
10.1 I/O Systems	25
10.2 Disk Structure and Scheduling	25
11. Quick Revision / Exam Pointers	27

Operating System — Complete Notes

This guide builds Operating System concepts progressively for the **IBPS SO (IT Officer)** examination — from absolute basics to advanced numericals. Every classification, formula, Gantt chart, and examiner trap relevant to the MCQ paper is covered to the atomic level.

1. Introduction to Operating Systems

1.1 Basics of an Operating System

1.1.1 Definition of an Operating System

An **Operating System (OS)** is **system software** that acts as an **intermediary** between the **user/application programs** and the **computer hardware**. It controls and coordinates the use of hardware among the various application programs.

The OS is the **first program** loaded by the **bootstrap loader** (from **ROM/firmware**) at power-on, and it remains resident in main memory; this always-resident core is called the **kernel**.

Memory Trick: "OS = **R.E.S.T.** controller" — manages **R**esources, provides an **E**nvironment, **S**chedules work, and **T**ranslates user requests to hardware.

1.1.2 Goals and Functions of an OS

The **two primary goals** of an OS are **convenience** (make the computer easy to use — user goal) and **efficiency** (use hardware resources efficiently — system goal). A secondary goal is the **ability to evolve**.

Core functions of an OS:

- **Process Management** — creation, scheduling, termination, synchronization of processes.
- **Memory Management** — allocation/deallocation of main memory, tracking usage.
- **File Management** — creation, deletion, directory structure, mapping files to storage.
- **I/O Device Management** — buffering, caching, spooling, device drivers.
- **Secondary-Storage Management** — free-space management, storage allocation, disk scheduling.
- **Protection and Security** — controlling access of processes/users to resources.
- **Networking** — managing communication in distributed/networked systems.
- **Command Interpretation** — the **shell/CLI** or **GUI** that interprets user commands.

1.1.3 OS as Resource Manager and Extended (Virtual) Machine

Two classic **views** of an OS:

- **Resource Manager (bottom-up view):** the OS allocates and de-allocates **CPU, memory, files, and I/O devices** among competing processes fairly and efficiently. It tracks who has what, grants requests, and resolves conflicts.
- **Extended / Virtual Machine (top-down view):** the OS hides the ugly hardware details (registers, interrupts, disk geometry) and presents programmers a clean, **abstract (extended) machine** that is easier to program through **system calls**.

MCQ Trap: The **top-down** view = **Extended/Virtual Machine** (abstraction); the **bottom-up** view = **Resource Manager** (allocation). Examiners swap these labels.

1.1.4 Components of an OS

- **Kernel** — the always-resident core managing CPU, memory, and devices.
- **Shell** — the command interpreter (interface between user and kernel); e.g., **Bash**, **PowerShell**.
- **File System** — organizes data on storage.
- **Device Drivers** — modules that control specific hardware devices.

- **System Libraries / System Programs** — utilities and APIs built atop the kernel.

1.1.5 System Calls

A **system call** is the **programmable interface** through which a user program requests a service from the OS kernel. A system call triggers a **mode switch** from **user mode to kernel mode** (via a **software interrupt / trap**).

Parameters are passed to the OS by **three methods**: via **registers**, via a **block/table in memory** (address passed in a register), or via the **stack** (push by program, pop by OS).

The **five categories** of system calls are:

Category	Purpose	Example Calls (UNIX / Windows)
Process Control	create, terminate, load, execute, wait	<code>fork()</code> , <code>exec()</code> , <code>exit()</code> , <code>wait()</code> / <code>CreateProcess()</code>
File Management	create, delete, open, read, write, close	<code>open()</code> , <code>read()</code> , <code>write()</code> , <code>close()</code>
Device Management	request, release, read/write devices	<code>ioctl()</code> , <code>read()</code> , <code>write()</code>
Information Maintenance	get/set time, date, system data, attributes	<code>getpid()</code> , <code>alarm()</code> , <code>sleep()</code>
Communication	create/delete connections, send/receive messages	<code>pipe()</code> , <code>shmget()</code> , <code>send()</code> , <code>recv()</code>
Protection	set permissions, allow/deny resource access	<code>chmod()</code> , <code>umask()</code> , <code>chown()</code>

MCQ Trap: `fork()` creates a **new (child) process** and returns **0 to the child** and the **child's PID to the parent**. `exec()` does **NOT** create a new process — it **replaces** the current process image.

1.1.6 System Programs

System programs (system utilities) provide a convenient environment for program development and execution; they sit **above** the kernel but **below** application software. Categories: **file management**, **status information**, **file modification (editors)**, **programming-language support (compilers, assemblers)**, **program loading/execution (loaders, linkers)**, and **communications**.

1.2 Operating System Structures

1.2.1 OS Architecture Types

Structure	Idea	Advantage	Disadvantage
Monolithic	Entire OS runs as a single program in kernel mode	Fast (direct calls), simple	Hard to maintain; one bug can crash all
Layered	OS split into N layers , each using services of the layer below (layer 0 = hardware)	Modular, easy to debug	Poor performance (many layers); hard to define layers
Microkernel	Keep only essentials (IPC, scheduling, low-level memory) in kernel; rest as user-mode servers	Reliable, secure, extensible	Slower due to message-passing overhead
Modular	Core kernel + loadable kernel modules (LKMs)	Flexible, dynamic	Module interfaces add complexity
Hybrid	Combines monolithic speed with microkernel modularity	Balanced	Complex design

Memory Trick: Microkernel = "Minimum in kernel." Examples: **Mach, QNX, L4, MINIX**. Hybrid examples: **Windows NT, macOS (XNU)**. Mostly Monolithic: **Linux, UNIX**.

1.2.2 Virtual Machines

A **Virtual Machine (VM)** provides an interface **identical to the bare hardware**, giving each VM the illusion of its own processor and memory. A **hypervisor (VMM — Virtual Machine Monitor)** creates and runs VMs.

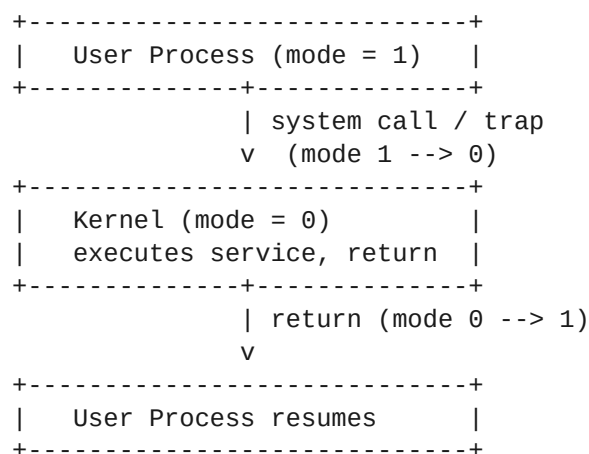
- **Type-1 (bare-metal) hypervisor** runs directly on hardware — e.g., **VMware ESXi, Xen, Hyper-V**.
- **Type-2 (hosted) hypervisor** runs on a host OS — e.g., **VMware Workstation, VirtualBox**.

1.2.3 Dual-Mode Operation (Kernel vs User Mode)

To protect the OS, hardware supports **two modes** indicated by a **mode bit**:

- **User mode (mode bit = 1)**: runs user applications; **privileged instructions are NOT allowed**.
- **Kernel/Supervisor/Monitor mode (mode bit = 0)**: runs OS code; **all instructions allowed**.

A **privileged instruction** (e.g., I/O, halt, set timer, modify memory-management registers) executed in user mode causes a **trap** to the OS. Some CPUs (e.g., Intel) add intermediate **rings (0–3)**: **Ring 0 = kernel, Ring 3 = user**.



2. Types of Operating Systems

2.1 Classification of Operating Systems

2.1.1 Simple Batch Systems

Jobs with similar needs were **batched** together and run sequentially without user interaction; a **resident monitor** automatically transferred control from one job to the next. **No CPU–I/O overlap**, so CPU sits **idle during I/O**. Throughput improved over manual operation but turnaround was poor.

2.1.2 Multiprogrammed Batch Systems

Several jobs are kept in main memory at once; when one job waits for **I/O**, the CPU is switched to another **ready** job. This **maximizes CPU utilization** and introduces the need for **job scheduling, memory management, and CPU scheduling**.

MCQ Trap: Multiprogramming = keep CPU busy by switching among jobs (goal: **CPU utilization**).
Multitasking/Time-sharing = multiprogramming + **rapid switching** for **interactive** users (goal: **response time**).

2.1.3 Time-Sharing (Multitasking) Systems

The CPU is shared by **time slices (quanta)** so rapidly that each user feels they have a dedicated machine. Provides **interactivity, online file systems**, and requires **CPU scheduling, virtual memory, and protection**. Response time is typically kept **under ~1 second**.

2.1.4 Multiprocessor (Parallel / Tightly-Coupled) Systems

Two or more CPUs share the **bus, clock, memory, and peripherals**. Benefits: **increased throughput, economy of scale, increased reliability (graceful degradation / fault tolerance)**.

- **Symmetric Multiprocessing (SMP)**: each processor runs an **identical copy** of the OS; **peer** relationship; most modern systems (Windows, Linux).
- **Asymmetric Multiprocessing (AMP)**: a **master** processor controls the system and assigns work to **slave** processors.

2.1.5 Distributed Systems (Loosely-Coupled)

Independent computers with their **own local memory**, connected by a network, that appear to users as a **single coherent system**. Communication via **message passing**. Benefits: **resource sharing, computation speedup, reliability, communication**.

2.1.6 Real-Time Systems (RTOS)

Used when **rigid time constraints** govern processing. Correctness depends on **both result and the time** it is produced.

- **Hard Real-Time**: deadlines are **absolutely guaranteed** (e.g., **airbags, pacemakers, missile control**); secondary storage limited/absent.
- **Soft Real-Time**: critical tasks get **priority**; missing a deadline degrades quality but is **tolerable** (e.g., **multimedia streaming, video conferencing**).

2.1.7 Network, Embedded/Handheld, and Clustered Systems

- **Network OS (NOS)**: users are **aware** of multiple machines; provides file sharing and remote login (e.g., **Windows Server, Novell NetWare**). (Distributed OS hides the network; NOS exposes it.)
- **Embedded / Handheld OS**: run on devices with **limited memory/CPU/power** (e.g., **Android, iOS, FreeRTOS, VxWorks**).
- **Clustered Systems**: multiple **independent systems** (nodes) linked to work together via shared storage; **asymmetric** (one hot-standby) or **symmetric** clustering, for **high availability**.

MCQ Trap: Distributed OS --> user sees **one system** (transparency). **Network OS** --> user **knows** about and explicitly accesses separate machines.

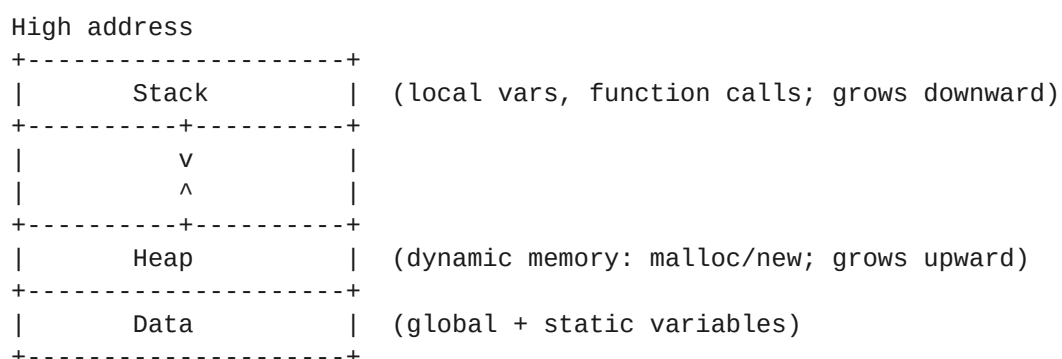
3. Process Management

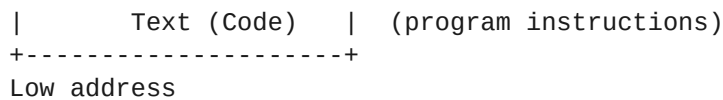
3.1 Process Concepts

3.1.1 Program vs Process

A **program** is a **passive** entity — an executable file stored on disk. A **process** is an **active** entity — a program **in execution**, with a program counter, registers, stack, and resources. One program can spawn **many processes**.

A process in memory has **four sections**:

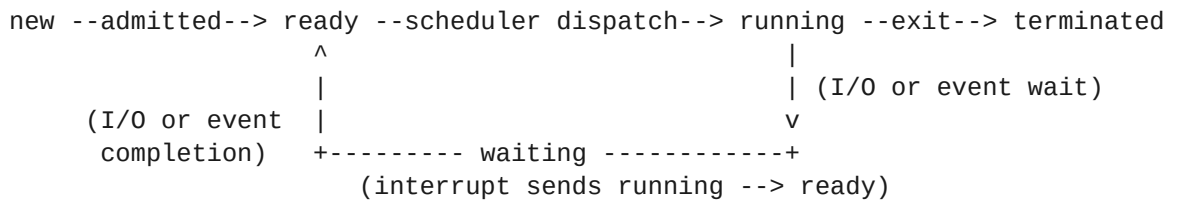




3.1.2 Process States and State-Transition Diagram

A process passes through **five states**:

- **New**: the process is being **created**.
- **Ready**: waiting to be **assigned to a CPU** (in main memory, ready to run).
- **Running**: instructions are being **executed** on the CPU.
- **Waiting / Blocked**: waiting for an **event (I/O completion, signal)**.
- **Terminated**: the process has **finished execution**.



Key transitions: **Ready --> Running** (dispatch by scheduler), **Running --> Ready** (timer interrupt/preemption), **Running --> Waiting** (I/O request), **Waiting --> Ready** (I/O complete).

MCQ Trap: There is **no direct Running --> Ready --> Waiting** chain for I/O. On an I/O request the process goes **Running --> Waiting** directly; only on **preemption/timeout** does it go **Running --> Ready**. Also **Waiting** transitions to **Ready** (never directly back to **Running**).

3.1.3 Process Control Block (PCB)

The **Process Control Block (PCB)** — also called **Task Control Block** — is the data structure that stores **all information about a process**. The OS maintains one PCB per process; together they form the **process table**.

PCB fields:

- **Process State** — new/ready/running/waiting/terminated.
- **Process ID (PID)** — unique identifier.
- **Program Counter (PC)** — address of the next instruction.
- **CPU Registers** — accumulators, index registers, stack pointers.
- **CPU-Scheduling Information** — priority, queue pointers, scheduling parameters.
- **Memory-Management Information** — base/limit registers, page tables, segment tables.
- **Accounting Information** — CPU time used, time limits, account numbers.
- **I/O Status Information** — list of open files, allocated I/O devices.

Memory Trick: PCB stores "**PP-R-MAI**" — **P**rocess state, **P**C, **R**egisters, **M**emory info, **A**ccounting, **I**/O status (+ PID & scheduling).

3.2 Process Scheduling

3.2.1 Scheduling Queues

- **Job Queue:** all processes in the system (on disk, waiting to enter memory).
- **Ready Queue:** processes in main memory, **ready and waiting to execute** (usually a linked list of PCBs).
- **Device / I/O Queue:** processes waiting for a particular **I/O device**.

3.2.2 Types of Schedulers

Scheduler	Also Called	Frequency	Controls	Speed
Long-Term	Job Scheduler	Infrequent (seconds–minutes)	Degree of multiprogramming (which jobs enter ready queue)	Slow
Short-Term	CPU Scheduler	Very frequent (milliseconds)	Which ready process gets the CPU	Fast
Medium-Term	Swapper	Medium	Swapping processes in/out of memory	Medium

The **long-term scheduler** controls the **degree of multiprogramming** (number of processes in memory) and aims for a good **mix of CPU-bound and I/O-bound** processes. The **medium-term scheduler** does **swapping** to reduce the degree of multiprogramming when memory is over-committed.

MCQ Trap: The **short-term scheduler** runs most frequently and **must be fast**. The **long-term scheduler** runs least frequently and controls the **degree of multiprogramming**. Time-sharing systems often have **no long-term scheduler** (use medium-term instead).

3.2.3 Context Switching

A **context switch** saves the state (PCB) of the currently running process and **restores** the state of the next scheduled process. Context-switch time is **pure overhead** (no useful work is done). Its cost depends on hardware (number of registers, presence of memory-management support). Hardware support like **multiple register sets** reduces this cost.

3.2.4 Dispatcher and Dispatch Latency

The **dispatcher** is the module that **gives CPU control** to the process selected by the short-term scheduler. It performs **context switch, switch to user mode, and jump** to the proper location in the user program. **Dispatch latency** is the time the dispatcher takes to **stop one process and start another**.

MCQ Trap: **Scheduler** *selects* the process (policy); **Dispatcher** *gives the CPU* to it (mechanism). Don't confuse **dispatch latency** with **scheduling**.

3.3 Threads

3.3.1 Thread Concept and Benefits

A **thread** is the **basic unit of CPU utilization** — a **lightweight process**. A thread has its **own** Thread ID, **program counter, register set, and stack**, but **shares** the **code, data section, and OS resources (open files, signals)** with other threads of the same process.

Benefits (4 R's): **Responsiveness**, **Resource sharing**, **Economy** (cheaper to create/switch than processes), and **Scalability** (parallelism on multiprocessors).

3.3.2 Process vs Thread

Aspect	Process	Thread
Weight	Heavyweight	Lightweight
Address space	Separate per process	Shared within a process
Creation cost	High	Low
Context switch	Slow (involves MMU)	Fast
Communication	IPC (costly)	Shared memory (easy)
Fault isolation	One crash does not kill others	One thread crash can kill the whole process

3.3.3 User-Level vs Kernel-Level Threads

- **User-Level Threads (ULT):** managed by a **user-space thread library**; kernel is unaware. **Fast** to create/manage, but **one blocking call blocks the whole process**, and **no true multiprocessor parallelism**.
- **Kernel-Level Threads (KLT):** managed by the **OS kernel**. Slower to manage, but **a blocking thread does not block others**, and they can run on **multiple CPUs**.

3.3.4 Multithreading Models

Model	Mapping (User : Kernel)	Notes
Many-to-One	many ULT : 1 KLT	One block stalls all; no parallelism (e.g., Green threads)
One-to-One	1 ULT : 1 KLT	True parallelism; per-thread overhead (Windows, Linux)
Many-to-Many	many ULT : many (<=) KLT	Best of both; flexible

Memory Trick: One-to-One = true parallelism but **costly**; Many-to-One = **cheap** but **no parallelism**; Many-to-Many = **balanced**.

3.4 Interprocess Communication (IPC)

Cooperating processes need to exchange data via **two fundamental models**:

3.4.1 Shared Memory Model

A region of memory is **shared** by cooperating processes; they exchange data by **reading/writing** this region. **Fast** (after setup, no kernel involvement), but the **programmer must handle synchronization** (race conditions).

3.4.2 Message Passing Model

Processes communicate by exchanging **messages** via **send()** and **receive()**; the **kernel manages** the channel. Useful in **distributed** systems; **slower** (kernel involvement) but **no synchronization conflicts** for shared data.

- **Direct communication:** name the recipient/sender explicitly; **one link per pair**.
- **Indirect communication:** messages sent to/from **mailboxes (ports)**; one mailbox can be shared.
- **Synchronous (blocking)** vs **Asynchronous (non-blocking)** send/receive.
- **Buffering:** zero capacity (rendezvous), bounded capacity, unbounded capacity.

Feature	Shared Memory	Message Passing
Speed	Faster (direct access)	Slower (kernel mediation)
Kernel involvement	Only at setup	Every message
Synchronization	Programmer's responsibility	Implicit in send/receive
Best for	Single machine, large data	Distributed systems, small data

3.4.3 Producer–Consumer Problem and IPC Mechanisms

The **Producer–Consumer (Bounded-Buffer)** problem is the canonical shared-memory cooperation example: a producer fills a buffer and a consumer empties it; they must not overwrite/underflow.

Other IPC mechanisms: **Pipes** (unidirectional, related processes — ordinary pipe; **named pipe / FIFO** for unrelated processes), **Sockets** (endpoint for network communication, identified by **IP + port**), and **Signals** (asynchronous notification, e.g., **SIGKILL**, **SIGINT**).

4. CPU Scheduling

4.1 Basic Concepts

4.1.1 CPU-I/O Burst Cycle

Process execution alternates between a **CPU burst** (computation) and an **I/O burst** (waiting for I/O). CPU-bound programs have **long CPU bursts**; I/O-bound programs have **short CPU bursts**. CPU-burst durations form a typically **exponential / hyperexponential** distribution (many short, few long).

4.1.2 Preemptive vs Non-Preemptive Scheduling

- **Non-preemptive (cooperative):** once the CPU is given to a process, it keeps it until it **terminates** or **switches to waiting**. (e.g., FCFS, SJF, non-preemptive Priority).
- **Preemptive:** the CPU can be **forcibly taken** from a running process (timer/higher-priority arrival). Needs care for **shared data consistency**. (e.g., RR, SRTF, preemptive Priority).

MCQ Trap: Scheduling decisions happen at: (1) Running --> Waiting, (2) Running --> Ready, (3) Waiting --> Ready, (4) Termination. **Cases (1) and (4) only => non-preemptive**; including (2) or (3) => **preemptive**.

4.1.3 Scheduling Criteria

Criterion	Definition	Optimize
CPU Utilization	Fraction of time CPU is busy	Maximize
Throughput	Processes completed per unit time	Maximize
Turnaround Time (TAT)	Completion Time - Arrival Time	Minimize
Waiting Time (WT)	Turnaround Time - Burst Time = time spent in ready queue	Minimize
Response Time	Arrival --> first CPU response	Minimize

Key formulas:

Turnaround Time (TAT) = Completion Time (CT) - Arrival Time (AT)

Waiting Time (WT) = Turnaround Time - Burst Time (BT)

Response Time = First CPU allocation time - Arrival Time

Average WT = (sum of all WT) / number of processes

MCQ Trap: Waiting Time = TAT - Burst Time (NOT Completion - Burst unless arrival = 0). **Response time** counts only until the **first** CPU response, whereas TAT counts until **completion**.

4.2 Scheduling Algorithms (with Gantt Charts)

4.2.1 FCFS — First Come First Serve

The process that **requests the CPU first** is allocated first; implemented with a **FIFO queue**. **Non-preemptive**.

Example — all arrive at t=0; Burst: P1=24, P2=3, P3=3:

Gantt: | P1 | P2 | P3 |
 0 24 27 30

- WT: P1=0, P2=24, P3=27 --> **Average WT = (0+24+27)/3 = 17 ms**
- TAT: P1=24, P2=27, P3=30 --> **Average TAT = 81/3 = 27 ms**

Convoy Effect: short processes wait behind a long one, lowering CPU and device utilization. FCFS gives **high average waiting time** and suffers the convoy effect.

MCQ Trap: FCFS is **always non-preemptive** and can cause the **convoy effect**. It is **not** optimal for average waiting time.

4.2.2 SJF — Shortest Job First (and SRTF)

Allocate the CPU to the process with the **smallest next CPU burst**. **SJF is provably optimal** — it gives the **minimum average waiting time** for a given set of processes.

- **Non-preemptive SJF**: once started, a job runs to completion.
- **Preemptive SJF = SRTF (Shortest Remaining Time First)**: a newly arrived process with a **shorter remaining time** preempts the running one.

Non-preemptive SJF — all arrive $t=0$; Burst: P1=6, P2=8, P3=7, P4=3:

Gantt: | P4 | P1 | P3 | P2 |
0 3 9 16 24

- WT: P4=0, P1=3, P3=9, P2=16 --> **Average WT = $28/4 = 7$ ms**

SRTF (preemptive) — Arrival/Burst: P1(0/8), P2(1/4), P3(2/9), P4(3/5):

Gantt: | P1 | P2 | P4 | P1 | P3 |
0 1 5 10 17 26

- WT: P1=9, P2=0, P3=15, P4=2 --> **Average WT = $26/4 = 6.5$ ms**

MCQ Trap: SJF needs the **next burst length in advance** (unknown!) — it is predicted via **exponential averaging**: $\tau(n+1) = \alpha \cdot t(n) + (1-\alpha) \cdot \tau(n)$. SJF can cause **starvation** of long processes.

4.2.3 Priority Scheduling

Each process has a **priority**; the CPU goes to the **highest-priority** process. **Smaller integer usually = higher priority** (convention varies). Can be **preemptive** or **non-preemptive**. SJF is a special case (priority = inverse of next burst).

Non-preemptive example — all $t=0$; (Burst, Priority[1=high]): P1(10,3), P2(1,1), P3(2,4), P4(1,5), P5(5,2):

Gantt: | P2 | P5 | P1 | P3 | P4 |
0 1 6 16 18 19

- WT: P2=0, P5=1, P1=6, P3=16, P4=18 --> **Average WT = $41/5 = 8.2$ ms**

Starvation (indefinite blocking): low-priority processes may never run. **Solution: Aging** — gradually **increase priority** of waiting processes over time.

MCQ Trap: Aging is the standard cure for **starvation** in priority scheduling. A classic anecdote: an MIT shutdown found a low-priority job submitted in **1967** still waiting.

4.2.4 Round-Robin (RR)

Designed for **time-sharing**: each process gets a fixed **time quantum (q)**; after q expires it is **preempted** and moved to the **tail of the ready queue**. **Preemptive**, FIFO-based, **starvation-free**.

Example — all $t=0$; Burst: P1=24, P2=3, P3=3; **quantum q = 4**:

Gantt: | P1 | P2 | P3 | P1 | P1 | P1 | P1 | P1 |
0 4 7 10 14 18 22 26 30

- WT: P1=6, P2=4, P3=7 --> **Average WT = $17/3 \sim 5.66$ ms**

Effect of quantum:

- **q --> infinity** : RR degenerates to **FCFS**.
- **q --> very small** : behaves like **processor sharing** but **context-switch overhead dominates**.
- Rule of thumb: **80%** of CPU bursts should be **shorter than the quantum**.

MCQ Trap: If quantum is **q** and there are **n** processes, each waits **no longer than $(n-1) \times q$** for its next turn. Large q --> FCFS; tiny q --> high context-switch overhead.

4.2.5 Multilevel Queue (MLQ) Scheduling

The ready queue is **partitioned into separate queues** (e.g., **system, interactive, batch**), each with its **own scheduling algorithm**. A process is **permanently assigned** to one queue. Scheduling **between queues** is usually **fixed-priority preemptive** (or time-slice among queues).

4.2.6 Multilevel Feedback Queue (MLFQ) Scheduling

Like MLQ, but processes **can move between queues** based on behavior (**aging/demotion**). A process using too much CPU is **demoted** to a lower-priority queue; one waiting long is **promoted** (aging). It is the **most general** and **most complex** scheduler, defined by: **number of queues, scheduling per queue, promotion/demotion rules, and entry queue**.

MCQ Trap: MLQ — no movement between queues (fixed). **MLFQ — processes move** between queues (feedback). MLFQ can prevent starvation via **aging**.

4.2.7 HRRN — Highest Response Ratio Next

A **non-preemptive** algorithm that selects the process with the **highest response ratio**, balancing short jobs and waiting time (reduces starvation of SJF):

Response Ratio (RR) = (Waiting Time + Burst Time) / Burst Time

A long-waiting process eventually wins (numerator grows), so HRRN **avoids starvation** while still favoring short jobs.

Algorithm	Preemptive?	Optimizes	Starvation?
FCFS	No	Simplicity	No (but convoy effect)
SJF	No	Min avg WT	Yes (long jobs)
SRTF	Yes	Min avg WT	Yes
Priority	Both	Importance	Yes (cure: aging)
Round-Robin	Yes	Response time	No
HRRN	No	Balanced	No

4.3 Advanced Scheduling

4.3.1 Multiple-Processor Scheduling

- **Asymmetric multiprocessing**: one **master** processor handles all scheduling; others run user code.
- **Symmetric multiprocessing (SMP)**: each processor schedules **itself** from a common or per-processor ready queue.
- **Processor Affinity**: keep a process on the **same processor** to reuse its **cache** — **soft affinity** (preference) vs **hard affinity** (guaranteed).
- **Load Balancing**: keep work even across CPUs — **push migration** (a task moves overloaded --> idle) and **pull migration** (idle CPU pulls work).

4.3.2 Real-Time Scheduling

- **Rate-Monotonic (RMS)**: **static** priority — **shorter period => higher priority**. Optimal among fixed-priority algorithms; CPU utilization bound for **n** tasks is $U \leq n(2^{1/n} - 1)$ (--> **ln 2 ~ 69.3%** as $n \rightarrow \infty$).
- **Earliest-Deadline-First (EDF)**: **dynamic** priority — the task with the **earliest deadline** runs first; theoretically achieves **100%** CPU utilization.

MCQ Trap: RMS = static priority (period-based), schedulable up to **~69%** for many tasks; **EDF = dynamic** priority (deadline-based), up to **100%** utilization.

4.3.3 Thread Scheduling

On systems with kernel threads, the kernel schedules **threads**, not processes. **Contention scope: PCS (Process-Contention Scope)** schedules user threads within a process onto LWPs; **SCS (System-Contention Scope)** competes among **all threads system-wide** (used by one-to-one systems like Linux/Windows).

5. Process Synchronization

5.1 Synchronization Basics

5.1.1 Race Condition

A **race condition** occurs when **multiple processes/threads access and manipulate shared data concurrently**, and the **final result depends on the order** of execution. Example: two threads doing **counter++** (read–modify–write) can lose an update. Synchronization prevents races.

5.1.2 The Critical-Section Problem

A **critical section (CS)** is the code segment where a process **accesses shared resources**. The problem: design a **protocol** so that when one process is in its CS, **no other** is allowed in its CS.

```
do {
    [ entry section ]    // request permission
    critical section    // access shared data
    [ exit section ]    // release permission
    remainder section   // other code
} while (true);
```

5.1.3 Three Requirements of a Correct Solution

- **Mutual Exclusion:** at most **one** process in the critical section at a time.
- **Progress:** if no process is in its CS and some wish to enter, the selection of who enters **cannot be postponed indefinitely** (only contenders decide).
- **Bounded Waiting:** a **bound** exists on how many times other processes enter their CS **after** a process requests entry and **before** it is granted (no starvation).

MCQ Trap: The **three** requirements are **Mutual Exclusion, Progress, Bounded Waiting**. "Performance/efficiency" is **NOT** one of the three formal requirements.

5.2 Software and Hardware Solutions

5.2.1 Peterson's Solution (Software, 2 processes)

Uses **two shared variables**: **int turn** and **boolean flag[2]**. A process sets its **flag = true**, sets **turn** to the other, and waits while the other wants in and it is the other's turn. Peterson's solution **satisfies all three** requirements (ME, Progress, Bounded Waiting) but assumes **atomic load/store** and may fail on **modern reordering CPUs**.

```
// Process i (other = j)
flag[i] = true;
turn = j;
while (flag[j] AND turn == j) ; // busy wait
critical section
flag[i] = false;
remainder section
```

5.2.2 Dekker's Algorithm

The **first known correct software solution** to the two-process mutual-exclusion problem; like Peterson's it uses **flags + turn** but is older and more elaborate.

5.2.3 Hardware Support

- **Disabling Interrupts:** simplest on **uniprocessors** (prevents preemption), but **does not work on multiprocessors** and is **dangerous** (can hang the system).
- **Test-and-Set (TSL):** an **atomic** instruction that returns the old value of a lock and sets it to **true**.
- **Compare-and-Swap (CAS) / Swap:** atomically compares a value to an expected value and swaps if equal — basis of modern **lock-free** structures.

```
boolean TestAndSet(boolean *target) {
    boolean rv = *target;
    *target = true;
    return rv;           // atomic
}
// usage: while (TestAndSet(lock)) ; // spin
//         critical section
//         lock = false;
```

MCQ Trap: **Test-and-Set** and **Compare-and-Swap** guarantee **mutual exclusion** but **NOT bounded waiting** by themselves (a process can spin forever). Extra logic (a waiting array) is needed for bounded waiting.

5.3 Semaphores

5.3.1 Definition and Operations

A **semaphore S** is an **integer variable** accessed only via **two atomic operations**: **wait()** (also **P**, from Dutch **proberen** "to test") and **signal()** (also **V**, **verhogen** "to increment"). Invented by **Edsger Dijkstra**.

```
wait(S) {
    while (S <= 0) ;
    S = S - 1;
}
signal(S) {
    S = S + 1;
}
```

5.3.2 Counting vs Binary Semaphores

- **Counting semaphore:** value ranges over an **unrestricted domain**; used to control access to a resource with **N instances** (initialized to N).
- **Binary semaphore (mutex):** value is **0 or 1** only; provides **mutual exclusion** (like a lock).

5.3.3 Busy-Waiting (Spinlock) vs Block-Wakeup

- The simple definition above causes **busy waiting (spinlock)** — the process loops, wasting CPU. **Advantage:** no context switch (good for **short** critical sections on multiprocessors).
- The **better implementation** maintains a **waiting queue**: **wait()** **blocks** the process (**block()**), and **signal()** **wakes one** (**wakeup()**). Here S can be **negative**, and **|S| = number of waiting processes**.

MCQ Trap: With the block-wakeup implementation, a **negative** semaphore value's magnitude equals the **number of blocked processes**. **Binary semaphore = mutex**; **counting semaphore** counts available **resource instances**.

5.4 Classical Synchronization Problems

5.4.1 Bounded-Buffer (Producer–Consumer)

Three semaphores: **mutex = 1** (mutual exclusion on buffer), **empty = n** (count of empty slots), **full = 0** (count of full slots). Producer: **wait(empty); wait(mutex); ...add...; signal(mutex); signal(full)**. Consumer is symmetric.

MCQ Trap: Order matters! Producer does **wait(empty) then wait(mutex)**. Reversing to **wait(mutex) then wait(empty)** causes **deadlock**.

5.4.2 Readers–Writers Problem

Many processes read; writers need **exclusive** access. **First R-W**: no reader waits unless a writer has access (**writers may starve**). **Second R-W**: once a writer is ready, it gets priority (**readers may starve**). Uses `rw_mutex`, `mutex`, and a `read_count`.

5.4.3 Dining Philosophers Problem

5 philosophers, 5 forks (chopsticks), each needs **two** (left + right) to eat. If all pick the left fork simultaneously, **deadlock**. Fixes: allow **at most 4** at the table, pick up both **only if both available**, or **asymmetric** order (odd picks left first, even picks right first). It illustrates **deadlock and starvation** elegantly.

5.4.4 Sleeping Barber Problem

One **barber**, one **barber chair**, **N waiting chairs**. If no customers, the barber **sleeps**; an arriving customer wakes him or waits if a chair is free, else **leaves**. Demonstrates coordination using semaphores for **customers, barber**, and a **mutex**.

5.5 High-Level Synchronization Constructs

5.5.1 Critical Regions / Conditional Critical Regions

A **critical region** is a high-level language construct `region v when (B) S;` — shared variable `v` is accessed in statement `S` only when boolean condition `B` is true, with **automatic mutual exclusion**. Reduces programmer errors compared to raw semaphores.

5.5.2 Monitors

A **monitor** is an **abstract data type** that encapsulates shared data plus the procedures that operate on it, guaranteeing that **only one process is active inside the monitor at a time** (automatic mutual exclusion). It is **easier and safer** than semaphores.

5.5.3 Condition Variables

Inside a monitor, **condition variables** support `wait()` (suspend the caller) and `signal()` (resume one suspended process). Unlike semaphores, monitor `signal()` on an **empty** condition has **no effect** (it is not remembered). Signalling disciplines: **signal-and-wait** vs **signal-and-continue**.

MCQ Trap: Semaphore `signal()` is counted/remembered (increments the value). **Monitor condition-variable `signal()` is lost** if no process is waiting. This is a favourite distinction.

6. Deadlocks

6.1 Deadlock Concepts

6.1.1 Definition and System Model

A **deadlock** is a state where **a set of processes are each waiting for an event (resource release) that only another process in the set can cause** — so none ever proceeds. Resources have **types** (CPU, memory, files) and **instances**. Protocol: **Request --> Use --> Release**.

6.1.2 Four Necessary Conditions (Coffman Conditions)

ALL FOUR must hold simultaneously for a deadlock:

- **Mutual Exclusion**: at least one resource is held in a **non-sharable** mode.
- **Hold and Wait**: a process **holds** at least one resource and is **waiting** for others.
- **No Preemption**: a resource **cannot be forcibly taken**; released only voluntarily.
- **Circular Wait**: a **circular chain** of processes exists, each waiting for a resource held by the next.

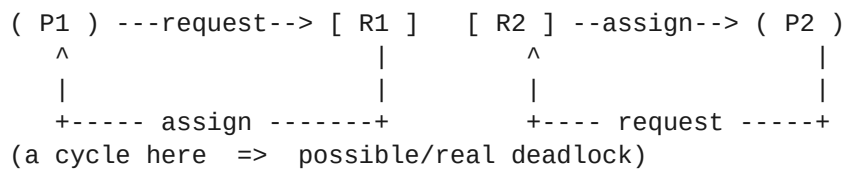
Memory Trick: Coffman conditions = "**MHC²**" wait — **M**utual exclusion, **H**old-and-wait, **n**o-preemption (**C**an't preempt), **C**ircular wait. **All four** are necessary; breaking **any one** prevents deadlock.

6.1.3 Resource-Allocation Graph (RAG)

A directed graph with **process nodes (circles)** and **resource nodes (squares with dots = instances)**.

- Request edge:** Process $\rightarrow$ Resource.
- Assignment edge:** Resource $\rightarrow$ Process.

Rules: **No cycle $\Rightarrow$ no deadlock**. If a cycle exists and each resource type has **one instance $\Rightarrow$ deadlock**. If resources have **multiple instances**, a cycle is **necessary but not sufficient** (may or may not be deadlock).



6.2 Deadlock Handling Methods

The **four** strategies: **Prevention, Avoidance, Detection & Recovery, and Ignorance (Ostrich algorithm — used by UNIX/Windows)**.

6.2.1 Deadlock Prevention

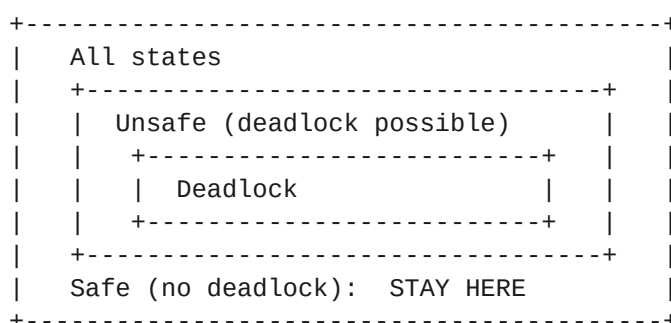
Ensure **at least one** Coffman condition **cannot hold**:

Condition Negated	Technique	Drawback
Mutual Exclusion	Make resources sharable (e.g., read-only)	Not possible for inherently non-sharable resources
Hold and Wait	Request all resources at once, or release before requesting	Low utilization, starvation
No Preemption	Preempt held resources if a request can't be met	Suited to CPU/memory, not printers
Circular Wait	Impose a total ordering of resource types; request in increasing order	Programmer discipline needed

6.2.2 Deadlock Avoidance

The system uses **a priori information** (maximum needs) to ensure it **never enters an unsafe state**.

- Safe State:** a sequence $\langle P_1, P_2, \dots, P_n \rangle$ exists where each P_i 's needs can be met by **currently available + resources held by all P_j ($j < i$)**. A safe state has **no deadlock**.
- Unsafe State:** may **lead to** deadlock (not guaranteed).
- RAG algorithm** (single instance): add a **claim edge** (dashed); grant a request only if converting it to an assignment edge **creates no cycle**.



MCQ Trap: Safe $\Rightarrow$ no deadlock. Unsafe does **NOT** imply deadlock (deadlock is only *possible*). Every **deadlock state is unsafe**, but not every unsafe state is a deadlock.

6.2.3 Banker's Algorithm

Used for **multiple instances** of resource types. Data structures (m types, n processes): **Available[m]**, **Max[n][m]**, **Allocation[n][m]**, and **Need[n][m] = Max - Allocation**.

Worked example — Available = **(3, 3, 2)**:

Process	Allocation (A B C)	Max (A B C)	Need (A B C)
P0	0 1 0	7 5 3	7 4 3
P1	2 0 0	3 2 2	1 2 2
P2	3 0 2	9 0 2	6 0 0
P3	2 1 1	2 2 2	0 1 1
P4	0 0 2	4 3 3	4 3 1

Safety Algorithm (find a safe sequence): start Work = Available = (3,3,2); find a process whose **Need ≤ Work**, "finish" it (Work += Allocation), repeat.

- P1: Need(1,2,2) ≤ (3,3,2) OK --> Work = (5,3,2)
- P3: Need(0,1,1) ≤ (5,3,2) OK --> Work = (7,4,3)
- P4: Need(4,3,1) ≤ (7,4,3) OK --> Work = (7,4,5)
- P0: Need(7,4,3) ≤ (7,4,5) OK --> Work = (7,5,5)
- P2: Need(6,0,0) ≤ (7,5,5) OK --> Work = (10,5,7)

Safe sequence = **<P1, P3, P4, P0, P2>** --> the state is **SAFE**.

Resource-Request Algorithm: when P_i requests Request: (1) if **Request ≤ Need[i]** and (2) **Request ≤ Available**, **pretend to allocate** and run the **safety check**; grant only if the resulting state is **safe**, else P_i waits.

MCQ Trap: Banker's needs the **maximum demand declared in advance**. Granting a request that leaves the system **unsafe** is **denied even if resources are physically available**. **Need = Max - Allocation** (not Max - Available).

6.2.4 Deadlock Detection

Allow deadlocks, then detect:

- **Single instance per type:** maintain a **wait-for graph** (collapse the RAG to processes only); a **cycle = deadlock**. Detection cost $O(n^2)$.
- **Multiple instances:** use a **detection algorithm** similar to Banker's safety (uses Allocation, Request, Available).

6.2.5 Deadlock Recovery

- **Process Termination:** abort **all** deadlocked processes, or abort **one at a time** until the cycle breaks (choose by priority, CPU time, resources held).
- **Resource Preemption:** **select a victim**, **roll back** it to a safe state, and prevent **starvation** (don't always pick the same victim — bound the number of rollbacks).

6.2.6 Starvation vs Deadlock

Aspect	Deadlock	Starvation
Definition	Circular wait; no process proceeds	A process waits indefinitely while others proceed
Cause	4 Coffman conditions	Resource policy/priority bias
Other processes	All blocked in the set	Others make progress
Also called	Circular wait	Indefinite blocking / livelock (variant)

MCQ Trap: In **deadlock NO process progresses**; in **starvation other processes DO progress** while the victim waits forever. Deadlock => starvation, but starvation does NOT imply deadlock.

7. Memory Management

7.1 Background

7.1.1 Logical vs Physical Address Space

- **Logical (virtual) address:** generated by the **CPU**; the set of all logical addresses = **logical address space**.
- **Physical address:** address seen by the **memory unit (RAM)**; the set = **physical address space**.
- The **Memory-Management Unit (MMU)** maps logical --> physical at **run time** (e.g., adds the **relocation/base register**). Logical and physical addresses are **identical** at compile/load time but **differ** at execution time.

7.1.2 Address Binding

Binding Time	When	Relocatable?
Compile time	Absolute addresses fixed in code	No — must recompile if location changes
Load time	Relocatable code; addresses bound at load	Only by reloading
Execution time	Binding deferred to run; needs MMU	Yes — process can move during execution

7.1.3 Dynamic Loading and Dynamic Linking

- **Dynamic Loading:** a routine is **loaded only when called** --> better memory use for rarely used code; no special OS support needed (program-controlled).
- **Dynamic Linking:** linking of **shared libraries (DLLs / .so)** is **postponed to execution time** using a **stub**; saves memory/disk (one copy shared). **Static linking** copies the library into each executable.

7.1.4 Swapping

A process can be **swapped out** of memory to a **backing store (disk)** and later **swapped in** to continue. Increases the **degree of multiprogramming** beyond physical memory. **Swap time is dominated by transfer time** (proportional to memory swapped). Modern systems use swapping only under memory pressure (paging variant).

7.2 Contiguous Memory Allocation

7.2.1 Partitioning Schemes

- **Single partition:** one user process at a time (besides OS).
- **Fixed (static) partitioning:** memory divided into **fixed-size** partitions; degree of multiprogramming bounded by partition count; suffers **internal fragmentation**.
- **Variable (dynamic) partitioning:** partitions created **to fit** each process; suffers **external fragmentation**.

7.2.2 Allocation Strategies (Dynamic Storage-Allocation Problem)

Strategy	Rule	Property
First Fit	Allocate the first hole big enough	Fastest ; good performance
Best Fit	Allocate the smallest hole that fits	Least leftover but most external fragmentation (tiny holes); slow (search all)
Worst Fit	Allocate the largest hole	Leaves large usable holes; worst in practice
Next Fit	First fit but resume from last position	Faster; spreads allocation

MCQ Trap: First Fit and Best Fit are better than Worst Fit in storage utilization and speed. Best Fit produces **smallest leftover holes** but is **slowest** and fragments memory into tiny unusable pieces.

7.2.3 Fragmentation and Compaction

- **Internal Fragmentation:** wasted space **inside** an allocated block (allocated - requested). Caused by **fixed-size** allocation/paging.
- **External Fragmentation:** total free memory is enough but it is **non-contiguous** (scattered holes). Caused by **variable-size** allocation/segmentation.
- **Compaction:** shuffle memory to place all free space **together**; requires **dynamic (execution-time) relocation**.
- The **50-percent rule:** with first fit, statistically **N allocated blocks** => **~0.5N lost** to fragmentation.

MCQ Trap: Paging --> Internal fragmentation. Segmentation / variable partition --> External fragmentation. Compaction cures external (not internal) fragmentation.

7.3 Paging

7.3.1 Pages and Frames

Physical memory is split into fixed-size **frames**; logical memory into same-size **pages** (typically **4 KB**, power of 2). Paging **eliminates external fragmentation** (a process need not be contiguous) but causes **internal fragmentation** in the **last page**.

7.3.2 Address Translation

A logical address has two parts: **page number (p)** + **page offset (d)**. The **page table** maps **p** --> **frame number (f)**; the physical address = $f \times \text{frame_size} + d$.

```

Logical Address: | page number p | offset d |
                  |
                  v (index page table)
Page Table:      [ p ] --> frame f
                  |
                  v
Physical Address: | frame f | offset d |

```

For a logical address space of 2^m and page size 2^n : high-order **m-n** bits = page number, low-order **n** bits = offset.

7.3.3 TLB and Effective Access Time (EAT)

The **Translation Look-aside Buffer (TLB)** is a small, fast **associative cache** of recent page-table entries. On a **TLB hit**, the frame is found quickly; on a **TLB miss**, the page table in memory is consulted (and the TLB updated).

$$\text{EAT} = h \times (\text{TLB_time} + \text{mem_time}) + (1 - h) \times (\text{TLB_time} + 2 \times \text{mem_time})$$

where **h = TLB hit ratio**. Example: TLB = 20 ns, memory = 100 ns, h = 0.80 --> EAT = $0.8 \times 120 + 0.2 \times 220$ = **140 ns**.

7.3.4 Multilevel and Inverted Page Tables

- **Multilevel (hierarchical) paging**: page the page table itself (2-level, 3-level) to avoid one huge contiguous table; increases translation memory accesses.
- **Inverted Page Table: one entry per physical frame** (not per page) --> table size proportional to **physical** memory, saving space, but **slower** lookup (search by pid + page); often paired with a **hash table**.
- **Shared Pages**: re-entrant (read-only) code can be **shared** among processes (e.g., one copy of an editor) by mapping the same frames.

7.4 Segmentation

7.4.1 Concept and Segment Table

Segmentation supports the **user view** of memory as a collection of **variable-size logical segments** (code, stack, heap, functions). A logical address = **<segment number s, offset d>**. The **segment table** holds, per segment, a **base** and a **limit**; the MMU checks $d < \text{limit}$ then computes $\text{base} + d$.

7.4.2 Segmentation with Paging

Combine both: segments are **paged** to remove external fragmentation while keeping the logical segment view (e.g., **Intel x86** uses segmentation + paging; **MULTICS**).

7.4.3 Paging vs Segmentation

Feature	Paging	Segmentation
Block size	Fixed (pages = frames)	Variable (logical segments)
Programmer view	Invisible (system view)	Visible (user/logical view)
Fragmentation	Internal	External
Address form	page number + offset	segment number + offset
Table	Page table (1 dimension)	Segment table (base + limit)
Sharing/Protection	Per-page (harder)	Easier (per logical segment)

8. Virtual Memory

8.1 Virtual Memory Concepts

8.1.1 Need and Benefits

Virtual memory allows execution of processes that are **not completely in main memory**, so a program can be **larger than physical memory**. Benefits: programs no longer constrained by RAM size, **more processes in memory** (higher multiprogramming, CPU utilization, throughput), and **less I/O** to load/swap programs. Implemented via **demand paging** or **demand segmentation**.

8.1.2 Demand Paging

A page is brought into memory **only when it is needed (referenced)** — a **lazy swapper (pager)**. A **valid-invalid bit** in the page table marks whether a page is in memory.

- **Pure demand paging**: start a process with **zero** pages in memory; fault in every page on first reference.
- **Pre-paging**: bring in pages **expected** to be needed soon, to reduce fault count.

8.1.3 Page Fault and Handling Steps

A **page fault** is a trap raised when a process references a page whose valid bit is **invalid (not in memory)**. Handling steps:

- **1)** Check the internal table — is the reference valid?
- **2)** If invalid --> terminate; if valid but not in memory --> continue.
- **3)** Find a **free frame** (or run page replacement to free one).
- **4)** **Schedule a disk read** to bring the page into the frame.
- **5)** Update the page table (set valid bit, frame number).
- **6)** **Restart** the instruction that faulted.

8.1.4 Effective Access Time with Page Faults

$$EAT = (1 - p) \times \text{memory_access_time} + p \times \text{page_fault_service_time}$$

where **p = page-fault rate** ($0 \leq p \leq 1$). Example: memory = 200 ns, page-fault service = 8 ms = 8,000,000 ns. If $p = 1/1000$ --> $EAT \sim 0.999 \times 200 + 0.001 \times 8,000,000 \sim \mathbf{8200\ ns}$ (~ 40× slowdown). This shows page-fault rate must be **extremely low**.

MCQ Trap: Even a **tiny** page-fault rate massively degrades performance because **disk access (ms)** is **~100,000× slower than RAM (ns)**. Keep **p** very small.

8.2 Page-Replacement Algorithms

When no free frame exists on a fault, a **victim page** is chosen and written back (if its **dirty/modify bit** is set) before being replaced. The **modify (dirty) bit** avoids writing back unmodified pages. Goal: **minimize the page-fault rate**. Below, reference string = 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1 with **3 frames**.

8.2.1 FIFO — First-In First-Out

Replace the **oldest** loaded page (a FIFO queue of pages). Simple but can replace a heavily used page. For the string above (3 frames): **15 page faults**.

8.2.2 Belady's Anomaly

For some algorithms (**notably FIFO**), **increasing the number of frames can INCREASE the number of page faults** — counter-intuitive! Classic demonstration with string 1 2 3 4 1 2 5 1 2 3 4 5:

- **3 frames --> 9 faults; 4 frames --> 10 faults** (more frames, more faults!).

MCQ Trap: Belady's anomaly affects FIFO (and some others). **Stack algorithms** (LRU, OPT/LFU-MFU as stack-type) **never** suffer Belady's anomaly because their frame sets are **subset-nested**.

8.2.3 Optimal (OPT / MIN)

Replace the page that **will not be used for the longest time in the future**. Gives the **lowest possible** fault count but is **unrealizable** (needs future knowledge) — used only as a **benchmark**. For the string above (3 frames): **9 page faults**.

8.2.4 LRU — Least Recently Used

Replace the page **not used for the longest time in the past** (approximates OPT looking backward). Implementations: **counters/timestamps** or a **stack** of page numbers. For the string above (3 frames): **12 page faults**. **LRU is a stack algorithm** (no Belady's anomaly).

8.2.5 LRU Approximation — Second-Chance (Clock)

Uses a **reference bit** per page. Pages are scanned in FIFO (circular **clock**) order; if a page's reference bit is **1**, give it a **second chance** (clear bit to 0, move on); if **0**, replace it. **Enhanced second-chance** uses the **(reference, modify)** pair to prefer evicting **unmodified** pages.

8.2.6 Counting-Based — LFU and MFU

- **LFU (Least Frequently Used):** replace the page with the **smallest reference count** (assumes rarely used = not needed). Problem: a page used heavily then never again stays in memory.
- **MFU (Most Frequently Used):** replace the page with the **largest count** (assumes the smallest count just arrived and will be used).

Algorithm	Replaces	Belady's Anomaly	Realizable?
FIFO	Oldest arrival	Yes	Yes
Optimal (MIN)	Used farthest in future	No	No (needs future)
LRU	Used farthest in past	No (stack)	Yes (costly)
Second-Chance/Clock	FIFO + reference bit	Approx.	Yes (cheap)
LFU	Smallest use count	Possible	Yes
MFU	Largest use count	Possible	Yes

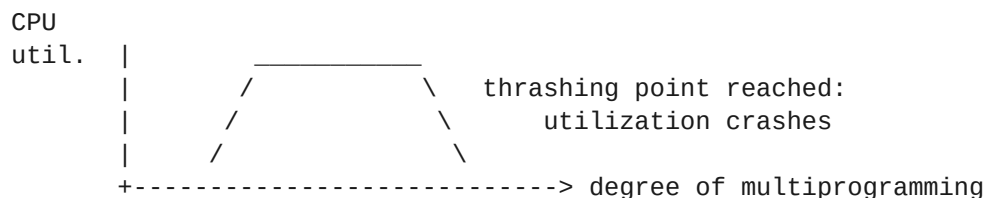
8.3 Frame Allocation and Thrashing

8.3.1 Frame Allocation

- **Equal allocation:** each of **n** processes gets **m/n** frames.
- **Proportional allocation:** frames given **in proportion to process size** (or priority).
- **Global replacement:** a process may take a frame from **any** process (common, better throughput, but a process can't control its own fault rate).
- **Local replacement:** a process replaces only **its own** frames (more predictable).

8.3.2 Thrashing

Thrashing is **high paging activity** where a process spends **more time paging than executing** — CPU utilization **collapses**. Cause: too high a **degree of multiprogramming** --> each process has **too few frames** --> constant faults.



Cure: reduce the **degree of multiprogramming**, use **local replacement**, and the **working-set / PFF** models to give each process enough frames.

8.3.3 Working-Set Model

Based on **locality of reference**. The **working set** is the set of pages referenced in the most recent **Delta (working-set window)** references. Sum of working-set sizes = **D** (total demand). If **D > total frames m** => **thrashing**; the OS suspends a process if demand exceeds supply.

8.3.4 Page-Fault Frequency (PFF)

A **direct** control of thrashing: measure each process's **page-fault rate** and keep it within an **upper and lower bound**. If rate **above upper bound** --> **give more frames**; if **below lower bound** --> **take frames away**.

MCQ Trap: Working set controls thrashing **indirectly** (locality window Delta); **PFF** controls it **directly** (fault-rate thresholds). Thrashing is fixed by **decreasing** the degree of multiprogramming, not increasing it.

9. File Management

9.1 File Concepts

9.1.1 File Attributes and Operations

A **file** is a **named collection of related information** on secondary storage — the logical storage unit abstracting physical device properties.

- **Attributes: Name, Identifier (unique number), Type, Location, Size, Protection, Timestamps (create/modify/access), Owner.**
- **Operations (6 basic): create, write, read, reposition (seek), delete, truncate** (open/close manage the system-wide & per-process **open-file tables**).

9.1.2 File Access Methods

Method	Description	Best for
Sequential	Records accessed in order , one after another (read next/write next)	Tapes, logs, editors
Direct / Relative	Access any block directly by block number (random)	Databases
Indexed	An index maps keys to block addresses; search index then access block	Large random-access files

9.2 Directory Structure

Structure	Description	Limitation
Single-Level	All files in one directory	Naming conflicts; no grouping
Two-Level	One directory per user (UFD + MFD)	No grouping within a user; limited sharing
Tree	Hierarchical; subdirectories	No sharing/links of same file
Acyclic Graph	Allows shared subdirs/files via links (no cycles)	Dangling pointers on delete
General Graph	Allows cycles	Needs garbage collection ; infinite loops

MCQ Trap: Acyclic-graph allows sharing but **NO** cycles; general-graph allows cycles but requires cycle detection / garbage collection. A **tree** structure does **not** allow shared files.

9.3 File Allocation Methods

Method	How	Pros	Cons
Contiguous	File occupies consecutive blocks ; needs start + length	Fast sequential & direct access	External fragmentation ; file can't grow easily
Linked (FAT)	Each block points to the next ; directory has first/last block	No external fragmentation; easy growth	No efficient direct access ; pointer overhead; reliability (lost pointer)
Indexed	An index block holds all block pointers	Supports direct access ; no external fragmentation	Index block overhead; size limits

- **Linked variant — FAT (File Allocation Table):** the pointers are gathered in a table at the start of the volume (used by **MS-DOS / Windows**).
- **Indexed variants: single-level index, multilevel index** (index of indexes), and **combined scheme** — as in **UNIX inode: direct blocks + single, double, and triple indirect** pointers.

MCQ Trap: The **UNIX inode** uses a **combined** scheme: **12 direct + 1 single-indirect + 1 double-indirect + 1 triple-indirect** block pointer. Contiguous = external fragmentation; Linked = poor random access.

9.4 Free-Space Management

Technique	Idea	Note
Bit Vector (Bitmap)	1 bit per block (1 = free, 0 = used)	Simple; easy to find contiguous runs; bitmap must be in memory
Linked List	Free blocks chained together	No wasted space; no easy contiguous allocation
Grouping	First free block stores addresses of n free blocks	Fast to find many free blocks
Counting	Store address + count of contiguous free blocks	Compact when free blocks are clustered

9.5 Protection and Access Control

Protection controls **who** can do **what** to files. Mechanisms:

- **Access Matrix:** rows = **domains**, columns = **objects**; entry = allowed **rights**.
- **Access Control List (ACL):** per-object list of (user, rights) — a **column** of the matrix.
- **Capability List:** per-domain list of objects + rights — a **row** of the matrix.
- **UNIX permissions:** **3 classes** (owner, group, others) × **3 rights** (**read=4, write=2, execute=1**) --> e.g., `chmod 755 = rwxr-xr-x`.

MCQ Trap: **ACL = column** of the access matrix (per object). **Capability list = row** (per domain). `chmod 755`: owner **rwx(7)**, group **r-x(5)**, others **r-x(5)**.

10. I/O and Disk Management

10.1 I/O Systems

10.1.1 I/O Techniques

- **Programmed I/O / Polling (busy-waiting):** CPU repeatedly checks the device **status (busy bit)** — wastes CPU.
- **Interrupt-driven I/O:** device **interrupts** the CPU when ready — CPU does other work meanwhile.
- **Direct Memory Access (DMA):** a **DMA controller** transfers a whole block **directly between device and memory**, interrupting the CPU **only once per block** — frees the CPU; uses **cycle stealing**.

MCQ Trap: **DMA** transfers data **without per-byte CPU involvement** and raises **one interrupt per block** (not per byte/word). Polling wastes CPU; interrupts and DMA free it.

10.1.2 Buffering and Spooling

- **Buffering:** store data in memory while transferring, to cope with **speed mismatch** and **size mismatch** between producer and consumer.
- **Caching:** keep a copy of frequently used data in faster storage.
- **Spooling (Simultaneous Peripheral Operation On-Line):** buffer output for a device that can serve **one job at a time** (e.g., **printer**); jobs queued on disk and printed in order.

10.2 Disk Structure and Scheduling

10.2.1 Disk Structure and Performance

A magnetic disk has **platters** --> **tracks** --> **sectors**; the same track across platters forms a **cylinder**. Disk access time has three parts:

- **Seek Time:** move the **arm/head** to the target **cylinder** (largest, most variable component).
- **Rotational Latency:** wait for the target **sector** to rotate under the head (~ **half a rotation** on average).

- **Transfer Time:** time to actually **transfer** the data.

Disk Access Time = Seek Time + Rotational Latency + Transfer Time

MCQ Trap: **Seek time** is usually the **dominant, most-optimized** component — that is why disk scheduling targets head movement. Average **rotational latency** = $(1/2) \times (1/\text{RPM}) \times 60$ seconds.

10.2.2 Disk-Scheduling Algorithms

Goal: minimize **total head movement**. Standard example — request queue **98, 183, 37, 122, 14, 124, 65, 67**, head starts at **53**, disk cylinders **0–199**, arm moving toward **higher** numbers.

Algorithm	Service Order (head 53)	Total Head Movement
FCFS	98,183,37,122,14,124,65,67	640
SSTF	65,67,37,14,98,122,124,183	236
SCAN	up to 199 (end), then 37,14	331
C-SCAN	up to 199, jump to 0, then 14,37	382
LOOK	up to 183, reverse to 37,14	299
C-LOOK	up to 183, jump to 14, then 37	322

- **FCFS:** serve in arrival order — fair but **poor** (wild swings).
- **SSTF (Shortest Seek Time First):** serve the **nearest** request next — better, but **starvation** of far requests.
- **SCAN (Elevator):** head moves to **one end** servicing requests, then **reverses** — like an elevator.
- **C-SCAN (Circular SCAN):** services in **one direction** only; at the end **jumps back** to the start without servicing --> **more uniform wait**.
- **LOOK / C-LOOK:** like SCAN/C-SCAN but the arm **reverses at the last request**, not the physical disk end — **less movement**.

MCQ Trap: **SSTF can starve**; **SCAN/C-SCAN/LOOK/C-LOOK do not**. **C-SCAN gives more uniform wait** than SCAN. **LOOK != SCAN:** LOOK does **not** go to the physical disk end.

10.2.3 RAID Levels (0–6)

RAID (Redundant Array of Independent Disks) combines disks for **performance** and/or **reliability** via **striping, mirroring, and parity**.

Level	Technique	Fault Tolerance	Min Disks
RAID 0	Striping only	None (no redundancy)	2
RAID 1	Mirroring	1 disk per mirror	2
RAID 2	Bit-level striping + Hamming ECC	Yes (rarely used)	—
RAID 3	Byte-level striping + dedicated parity	1 disk	3
RAID 4	Block-level striping + dedicated parity	1 disk	3
RAID 5	Block striping + distributed parity	1 disk	3
RAID 6	Block striping + double distributed parity	2 disks	4

Memory Trick: **RAID 0 = speed (no safety)**, **RAID 1 = mirror**, **RAID 5 = distributed parity (survive 1 failure)**, **RAID 6 = double parity (survive 2)**. **RAID 0+1 / 1+0 (10)** combine striping and mirroring.

10.2.4 Disk Formatting, Boot Block, and Bad Blocks

- **Low-level (physical) formatting:** divides a platter into **sectors** (with header, data, ECC) before any data.
- **Logical formatting:** creates the **file system** (boot block, FAT/inodes, free lists).

- **Boot Block:** a small **bootstrap** program in a fixed location (e.g., **MBR — Master Boot Record** on sector 0) that loads the OS kernel.
- **Bad Blocks:** defective sectors handled by **sector sparing (forwarding)** — remap to spare sectors — or **sector slipping**.

11. Quick Revision / Exam Pointers

- **Process states:** New, Ready, Running, Waiting, Terminated — know all transitions.
- **PCB fields:** PID, state, PC, registers, scheduling info, memory info, accounting, I/O status.
- **Schedulers:** Long-term (degree of multiprogramming), Short-term (fast, frequent), Medium-term (swapping).
- **Scheduling formulas:** $TAT = CT - AT$, $WT = TAT - BT$, $RT = \text{first run} - AT$; **SJF = min avg WT (optimal)**.
- **Synchronization:** 3 requirements (ME, Progress, Bounded Waiting); semaphore **wait/signal**; **binary = mutex**.
- **Deadlock:** 4 Coffman conditions (all must hold); **Banker's** safe-sequence; **Need = Max - Allocation**.
- **Memory:** Paging --> internal fragmentation; Segmentation --> external; **compaction cures external**.
- **Virtual memory:** demand paging; $EAT = (1-p) \cdot ma + p \cdot pft$; **Belady's anomaly = FIFO**; **OPT < LRU < FIFO** faults (typically).
- **Disk:** $\text{Access} = \text{seek} + \text{rotational latency} + \text{transfer}$; **SSTF starves, SCAN/LOOK don't**.
- **Difference tables to memorize:** Paging vs Segmentation, Prevention vs Avoidance, Preemptive vs Non-preemptive, Process vs Thread, Deadlock vs Starvation, Internal vs External fragmentation.

Shortcut: Numerical practice priorities — **(1)** CPU scheduling (Gantt, AWT/ATAT), **(2)** Banker's safe sequence, **(3)** page-replacement fault counting, **(4)** disk-scheduling head movement, **(5)** EAT/TLB calculations. These five cover the bulk of OS numerical MCQs.